



8. Heap & Code con priorità



Lezione 17 - @October 31, 2025

Operazioni su heap

gli heap vengono utilizzati in vari contesti.

gli alberi quasi completi garantiscono

- altezza logaritmica
- implementazione con vettore posizionale

elemento di chiave massima

in un max heap, per definizione, guardo la radice, numero di passi $O(1)$

cancellare elemento di chiave massima

numero di passi $\Theta(\log n)$

inserimento di un nuovo elemento

aggiungo al primo posto libero (foglia) - es: aggiungi 40

nuova procedura: **risistema** "dal basso"

numero di passi $\Theta(\log n)$

cancella un elemento di chiave x

- sostituisci x con l'ultima foglia (che viene eliminata)
- sia f la chiave dell'ultima foglia
 - se $f < x$ applica risistema dall'alto a partire dal nodo modificato
 - se $f > x$ applica risistema dal basso a partire dal nodo modificato

conoscendo la posizione del nodo da cancellare, numero di passi $\Theta(\log n)$

modifica della chiave di un nodo

$x \rightarrow x'$

simile all'operazione precedente:

- se $x' < x$ applica risistema dall'alto a partire dal nodo modificato
- se $x' > x$ applica risistema dal basso a partire dal nodo modificato

numero di passi $\Theta(\log n)$



GLI HEAP NON SONO ADATTI ALLA RICERCA



Lezione 18 - @November 3, 2025

Code con priorità

- collezioni di dati da cui gli elementi vengono prelevati secondo un criterio di priorità

- ogni elemento ha una chiave: chiave più bassa indica priorità maggiore

Operazioni

- `findMin()` restituisce l'elemento minimo senza rimuoverlo
- `deleteMin()` restituisce l'elemento minimo della coda e lo rimuove
- `insert(elemento e, chiave e)` inserisce nella coda un elemento e con associata una chiave (priorità) k
- `delete(elemento e)`
- `changeKey(elemento e, chiave d)` modifica la priorità e , assegnando come nuovo valore d

→ implementazione tramite **MinHeap**

operazione	funzione heap	#passi
<code>findMin</code>	controlla radice	$O(1)$
<code>deleteMin</code>	<code>risistema</code>	$\Theta(\log n)$
<code>insert</code>	<code>risistemaDalBasso</code>	$\Theta(\log n)$
<code>delete</code>	<code>risistema</code> o <code>risistemaDalBasso</code>	$\Theta(\log n)$ #
<code>changeKey</code>	<code>risistema</code> o <code>risistemaDalBasso</code>	$\Theta(\log n)$ #

$\Theta(\log n)$ conoscendo la posizione dell'elemento da cancellare o modificare.

diversi livelli di astrazione:

- array posizionale
- min heap
- coda con priorità